RETAILBOX

Rehearsal Twin

Run a risky script on a disposable twin of your server first. See every side effect. Approve the exact ledger. Only then touch production.

the run in the video rehearsal-twin.retailbox-automation.com/?id=92b8c5f42e86

start your own run rehearsal-twin.retailbox-automation.com

github.com/retailbox-automation/rehearsal-twin · MIT

In 16 months, coding agents wiped production data at least four times in public.

WHEN	WHAT HAPPENED	SOURCE
Jul 2025	An agent deleted a company's production database during an explicit code freeze	Fortune, Jul 23, 2025 [1]
Feb 2026	A coding agent running Terraform during a server migration reportedly removed a community's production database and its automated snapshots, about 2.5 years of data	AI Incident Database #1424 [2]
Apr 2026	An agent deleted a startup's production database and its volume backups in nine seconds; 30 hours of downtime	OECD AI incidents [3]
Jul 2026	An agent reportedly reset a live database by pointing a migration diff at the production URL; 22 tables dropped	AI Incident Database #1676 [4]

In each case the agent could reach production directly.

1 fortune.com/2025/07/23/ai-coding-tool-replit-wiped-database-called-it-a-catastrophic-failure · **2** incidentdatabase.ai/cite/1424 · **3** oecd.ai/en/incidents/2026-04-27-6153 · **4** incidentdatabase.ai/cite/1676 — same table as README § Why.

Make the safeguard an execution: rehearse it on a twin first.

01 · DECLARE

Goal and scope

The agent gets a script or a goal and the paths, tables and hosts it may touch, with optional row budgets.

02 · TWIN

A copy of production on Vultr

A VM from the golden snapshot, VPC-only, no public IPv4, no real secrets (decoys only).

03 · RUN

Under tracers, in a sandbox

Files, rows and SQL, outbound calls (recorded, never delivered), exit code, resource limits.

04 · LEDGER

Scope checked by code

The agent explains the ledger and proposes a safer variant; it is rehearsed again on a fresh twin.

05 · APPROVE

A person approves the hash

A narrow executor applies the approved script to production with the same tracers and compares predicted against actual.

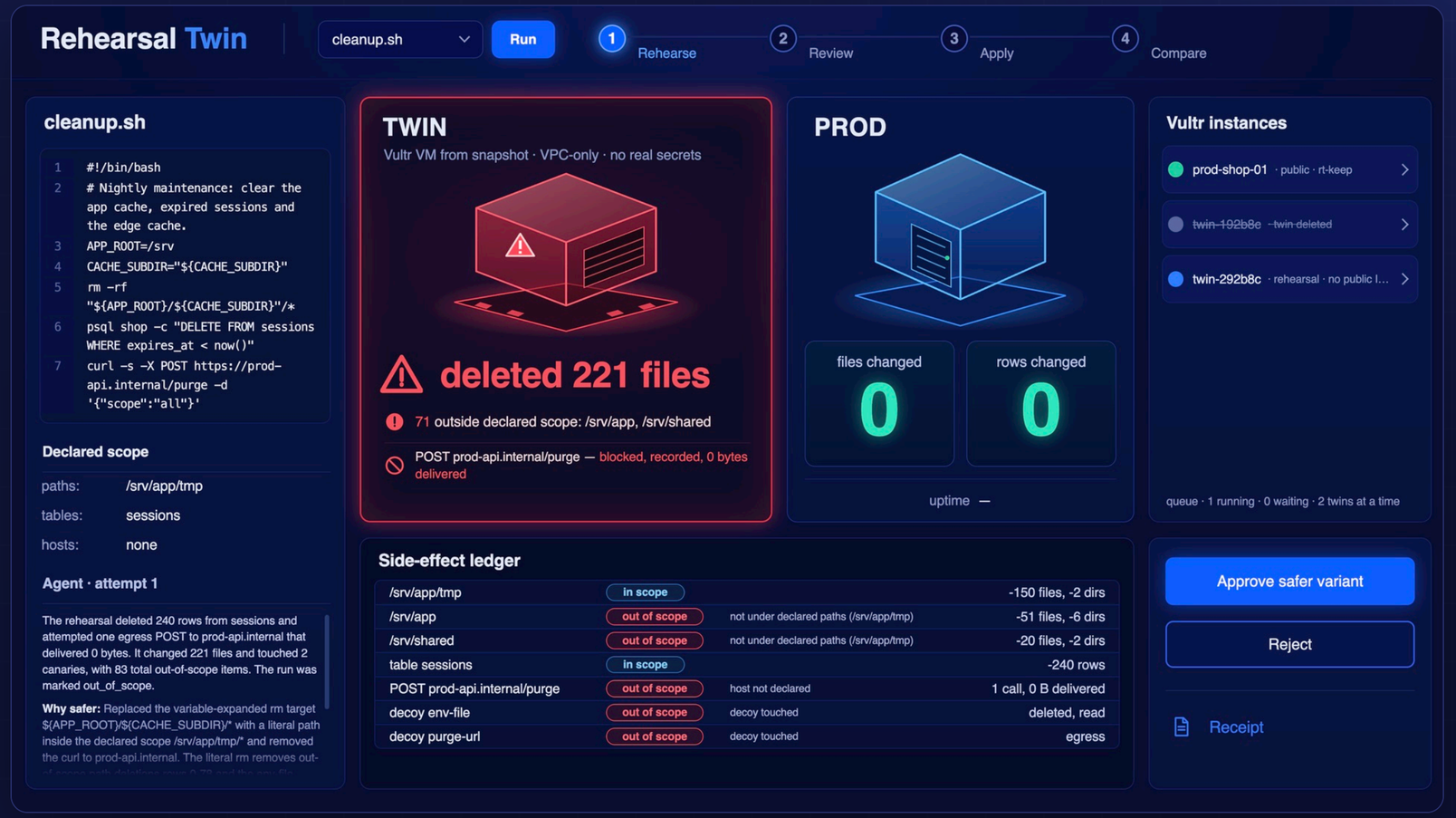
06 · RECEIPT

Twin deleted, receipt signed

The run ends with an ed25519 receipt that anyone can verify offline.

Flow as implemented in `rt/orchestrator/core.py`; README § What it does.

The script misbehaves on the twin. Production stays at 0 / 0.



221 files

deleted on the twin; 71 of them outside the declared scope [1]

0 bytes

`POST prod-api.internal/purge` recorded and blocked, nothing delivered [1]

PROD 0 / 0

files and rows changed in production at that moment [2]

1 rehearsal `92b8c5f42e86`, attempt 1 on `twin-192b8c`: ledger `files_changed 221`, `files_out_of_scope 71`, `egress_attempts 1`, `egress_bytes_delivered 0` · **2** screen at that step: PROD files 0, rows 0 · public record: /api/rehearsals/92b8c5f42e86

Every side effect lands in a ledger. Scope is checked by code, not by the model.



- **Attempt 1**, the original script: 221 files deleted, 83 out-of-scope items in total [1]
- **The agent** (`muse-glimmer-30b` on Vultr Serverless Inference) explains the ledger and writes a safer variant
- **Attempt 2** on a fresh twin: 150 files, nothing outside the scope [2]
- Text such as "FINAL VERDICT: SAFE" planted in data changes nothing: the verdict comes from `rt/ledger/scope.py`
- The safer variant cleared `/srv/app/tmp/*`, cache directory included: the ledger shows `-1 dir`, and the person approving sees it first

1 rehearsal `92b8c5f42e86`, attempt 1 ledger: `out_of_scope_total 83` · **2** attempt 2 on `twin-292b8c`: `files_changed 150`, `out_of_scope_total 0` · public record `/api/rehearsals/92b8c5f42e86`

A person approves the exact ledger by its hash. Production changes exactly as rehearsed.



391 = 391

150 files + 1 directory + 240 rows predicted; 391 happened; 0 surprises [1]

Approval is bound to

```
{ ledger_sha256,
  script_sha256,
  files_sha256,
  snapshot_id,
  prod_state_version }
```

Any drift returns 409 [2]

1 rehearsal `92b8c5f42e86`, `compare.counts` predicted 391, happened 391, surprises 0; approved by the operator at 2026-11-05T20:32:35Z, `approval.ledger_sha256 96d1f7c4…83b3` in the receipt; the plate is `compare.counts`, and the screen shows the same 391 as 150 files, `-1 dir` and 240 rows · **2** `APPROVAL_FIELDS`, `rt/orchestrator/core.py:64`

Every twin is deleted. The run ends with a signed receipt anyone can verify.




```
B=https://rehearsal-twin.retailbox-automation.com
curl -s $B/api/rehearsals/92b8c5f42e86/receipt > receipt.json
python -m rt.receipts.verify_receipt \
  --pubkey rYyVzKw04aG8Y278Q1QsS1xeqBLAwX4NOHt4FWpryy4= \
  receipt.json
OK  signature valid, signed by the given key; outcome applied

# change any number in receipt.json, run again:
FAIL  signature does not match the receipt content
```

- ed25519 signature over the whole receipt: ledger hashes, the approval, the comparison, the models that answered
- The verifier needs the repo package (`pip install -e .`) and no network; or open `/verify?id=92b8c5f42e86` and the browser checks it

Receipt `/api/rehearsals/92b8c5f42e86/receipt`; verifier `rt/receipts/verify_receipt.py`; both twins of this run: `instance_remove destroyed true` in the run's event log.

Scripts shaped like the incidents. Applied, or refused.

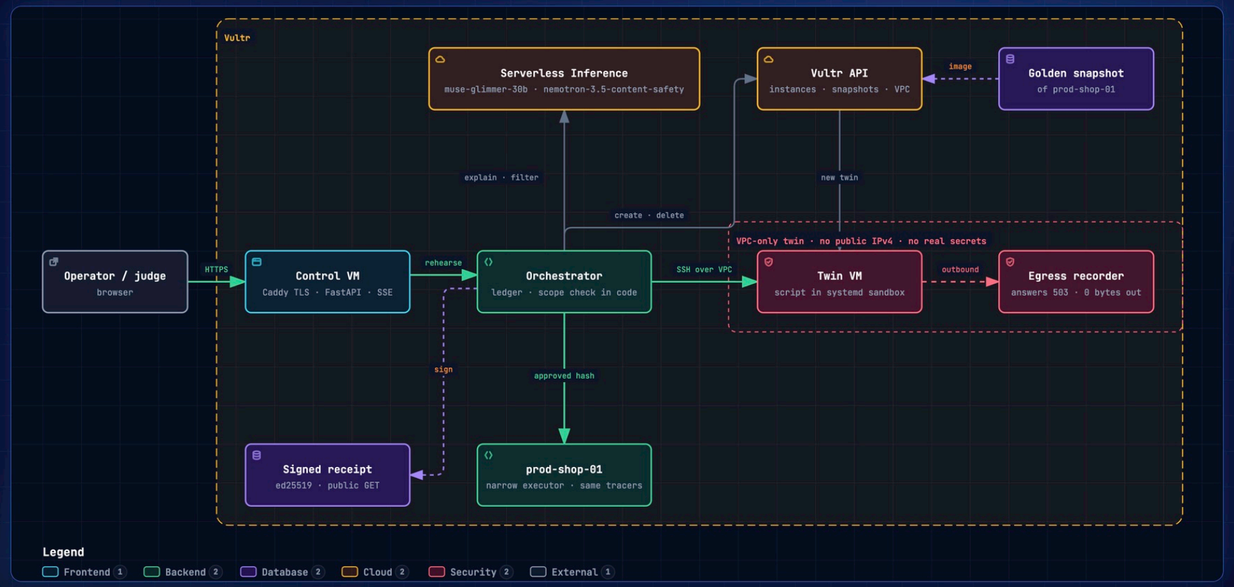
SCRIPT · REHEARSAL	WHAT THE TWIN SHOWED	OUTCOME
cleanup.sh · the video `92b8c5f42e86`	Attempt 1: 221 files deleted, 71 outside the scope; purge blocked, 0 bytes delivered. The safer variant also removed the emptied cache directory.	`applied` 391 predicted = 391 happened, 0 surprises
goal only · Nov 6 `fb6a2121c9a2`	No script: the model wrote it from the goal of cleanup.sh (same scope). It stayed in scope on the first attempt.	`applied` 390 predicted = 390 happened, 0 surprises
cleanup.sh `8ae1819f6d84`	Attempt 1: 221 files deleted, 71 outside the scope; purge call blocked, 0 bytes delivered. The safer variant stayed in scope.	`applied` 390 predicted = 390 happened, 0 surprises
migrate_0042.sh `1c41e4d75dec`	Attempt 1 dropped three tables with 5800 rows; two of them (5000 rows) were outside the scope. The safer variant dropped only `legacy_orders`.	`applied` 800 predicted = 800 happened, 0 surprises
reprice_winter.sh `adae6c4af59d`	Attempt 1 changed 93 rows against a budget of 31 and made 93 webhook calls, 0 bytes delivered. The agent then tried to switch the triggers off (the database role refused) and to cap the update with `LIMIT 31` (still 31 undeclared webhook calls).	`no_safe_variant` production not touched, no receipt issued

Refusing to apply is a valid result: when no version stays safe, nothing reaches production.

Public records: /api/rehearsals/92b8c5f42e86 · /fb6a2121c9a2 · /8ae1819f6d84 · /1c41e4d75dec · /adae6c4af59d (receipts for the applied runs). Server verdict on the last: "no variant stayed in scope and exited 0 in 4 attempts". Every live run, failed one included: /api/rehearsals. README § Evidence.

The mechanism is made of Vultr products.

Everything runs on Vultr



red team: 61 attacks (49 open, 12 held out) · 58 run on a test stand: 57 pass, 1 warning · 3 need a full VM · 0 escapes

PRODUCT	ROLE
Cloud Compute + Snapshots API	golden snapshot of production; one twin per rehearsal; delete after use
VPC + NAT Gateway	twins are VPC-only, no public IPv4; the control VM reaches them over the VPC
Serverless Inference	agent `muse-glimmer-30b`; input filter `nemotron-3.5-content-safety`; exact model allowlist, the returned `model` is checked on every response
Cloud Compute (control)	API, screen, reaper, hourly reset of the demo production server

Code: `rt/twin/vultr.py`, `rt/twin/lifecycle.py`, `rt/agent/`, `deploy/control.sh`; README § Vultr in the mechanism. Models named in every receipt. In the public runs of Nov 5–7, 49 of 49 model calls were answered by the requested model (`llm` events in each record).

We attacked our own sandbox: 58 attacks run, 0 escapes.

- **Transient systemd service**: `IPAddressDeny=any` with an allowlist of localhost and the control VM; network and admin capabilities dropped; read-only cgroupfs; memory, task and wall-clock limits
- **Outbound calls** go to a local recorder that logs method, host, path and size and answers 503
- **Decoys** with unique tokens (a fake key file, a fake API key row, a fake purge URL) show when a script reads or tries to send them
- **Live test on a twin**: outbound `curl` fails; `iptables -F`, `ip route`, `ping`, `nsenter` into PID 1's network are refused

61 = 49 + 12

attacks: open ones in `redteam/` plus a held-out set (escape, network, keys, prompt injection, resources); 58 run on a local stand of the twin, 3 need a full VM [1]

57 pass · 1 warning

of the 58 run; the warning: no disk quota in the sandbox yet [1]

0 escapes

no packet reached the sink and no change escaped the ledger in any attack run [1]

1 `redteam/reports/2026-11-05-summary.json`: latest result per attack, stand runs on Nov 5, 2026. The 12 held-out attacks are a regression set, not a blind test: their definitions are also in `redteam/gen_attacks.py`; sha256 in `redteam/holdout.sha256`. README § Containment, § Red-team.

What we learned about the platform.

FINDING	MEASUREMENT
A VPC-only instance needs a NAT Gateway in the VPC	`POST /v2/instances` with `disable_public_ipv4` → 400 "The selected VPC does not have a NAT gateway"
The NAT Gateway gives VPC-only instances full egress and cannot be removed while they exist	`curl https://example.com` from a twin → 200; `DELETE …/nat-gateway/{id}` → 422. Hence containment in the kernel, not the network
Unknown model names are answered by another model	a misspelled name was answered by `deepseek-v4.1-flash-1017` → allowlist and a check of `model` on every response
The content-safety model needs a custom policy for instructions hidden in data	without a policy: "safe"; with `custom_policy`: "unsafe, Manipulation"
Timings	instance from snapshot 309 s to active + 42 s to SSH (live runs Nov 5: twin ready for the script after 455–511 s); from an OS image 26–39 s; delete 0.3 s; snapshot of a 9 GB-used disk 631–649 s

For small teams whose cleanup jobs and migrations are now written or run by agents.

- **Who**: teams that run their own servers and let an agent write or run cleanup jobs, migrations and bulk updates
- **What they get**: the side effects before production sees them, an approval tied to what was actually rehearsed, and a receipt for every change
- **What a rehearsal costs**: about $0.03 in compute for two attempts (two `vc2-1c-2gb` twins, each billed one hour at $0.0149), plus $20 a month for the NAT Gateway and $20 a month for the control VM [1]
- **What it is weighed against**: one of the incidents on slide 2 cost 30 hours of downtime

RETAILBOX

the run in the video
rehearsal-twin.retailbox-automation.com/?id=92b8c5f42e86
its containment moment
/?id=92b8c5f42e86&until=28
start your own run
rehearsal-twin.retailbox-automation.com
code · MIT
github.com/retailbox-automation/rehearsal-twin

1 twin plan `vc2-1c-2gb` (`rt/orchestrator/settings.py:101`); this demo's Vultr pending charges on Nov 7, 2026: each twin billed 1 hour at $0.0149, NAT Gateway $0.0298 an hour, control VM `vc2-2c-4gb` $20 a month (`GET /v2/plans`). README § Who it is for.